

Introduction

To create a digital enhancement program, contrast adjustment using histogram equalisation on a cumulative intensity histogram was implemented, which back-projects the original image's intensities to create an image of equalised intensities. The following report details how the program works and showcases the enhanced images, as well as the results and run times of the optimisations made.

Implementation

All processing is performed using OpenCL kernels on the GPU, and my program is executed via a main function that accepts command-line arguments for platform selection (-p 0 or -p 1) to target the GPU or CPU respectively. The input files, which use .pgm or .ppm images, can directly be changed in the code, and this is what I have used to test when comparing both greyscale and colour images. These images are converted to 8-bit where necessary, and exception handling is implemented to catch both OpenCL and image loading errors.

OpenCL begins by selecting a platform and device, creating a context and building the program from the my_kernels.cl kernel file. Buffers are created dynamically (Figure 6) depending on image size and num_bins. This also includes input and output buffers, histograms, cumulative histograms and LUTs (lookup tables).

Greyscale images are processed directly, which proved effective in the results. For colour images, more configuration was needed, as the image is first converted to intensity using $Y = 0.299R + 0.587G + 0.114B$. Histogram equalisation is then applied to the Y channel only, and the resulting intensity is used to scale the original RGB values proportionally. This is meant to prevent colour distortion and allow perceptual enhancement, but I was unable to complete this, and the colour images remained noisy and unclear.

The .cl file contained my kernels, which included the histogram_kernel, which builds the histogram using atomic operations to avoid race conditions, and scan_kernel, which uses Hillis-Steele for cumulative histogram computation. normalise_kernel is the kernel that maps cumulative frequencies to the 0-255 range using min_bin and max_bin, which ensures the dynamic contrast enhancement. To apply the lookup tables, apply_lut_kernel and apply_lut_kernel_colour are used respectively. Each pixel in the original image serves as an index into the LUT, and the corresponding value is written into the output buffer. This mapping step effectively reassigns intensities based on the cumulative histogram, completing the equalisation process for the images.

Alongside support for colour images, I have implemented a number of features beyond the workshop exercises. The first of which is dynamic bin count, where the histogram size is configurable using the num_bins parameter. This is beneficial as it allows for flexibility and control over the contrast resolution depending on the image type and bit depth. 8-bit images typically use 256 bins (0-255), but 16-bit images may require up to 65,536 bins, and a dynamic bin count allows for the algorithm to scale accordingly without hardcoding for each case. Additionally, I have implemented minimum bin and maximum bin support, where the LUT is scaled based on the actual used intensity range. To measure the kernel and total time, I used OpenCL's cl::Event and chrono from C++. This allows for critical analysis, and is vital for optimising parallel operations.

Results

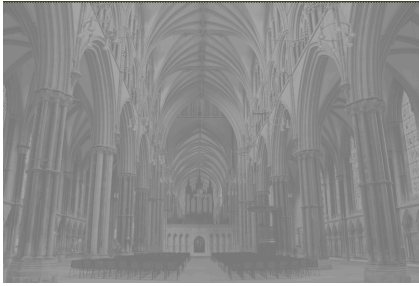


Figure 1: Original greyscale image



Figure 2: Greyscale image after enhancement



Figure 3: Original colour image

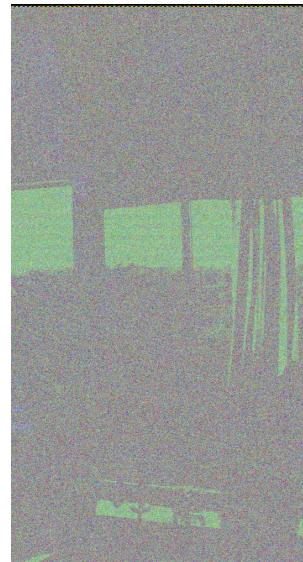


Figure 4: Colour image after enhancement

```

--- Execution Times ---
Memory Transfer (init): 0.001 ms
Histogram kernel:      4.451 ms
Scan kernel:          0.004 ms
Normalize kernel:     0.004 ms
Apply LUT kernel:     0.065 ms
Read buffer (output): 1.270 ms
Total Program Time:   203.859 ms

Histogram Equalisation complete. Output saved to output_equalised.pgm

C:\Users\Student\Downloads\parallel\x64\Debug\parallel.exe (process 13840) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|

```

Figure 5: Times taken in ms

```

cl::Buffer buf_input(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR, sizeof(unsigned char) * size, luminance.data());
cl::Buffer buf_output(context, CL_MEM_WRITE_ONLY, sizeof(unsigned char) * size);
cl::Buffer buf_hist(context, CL_MEM_READ_WRITE, sizeof(cl_uint) * num_bins);
cl::Buffer buf_scan(context, CL_MEM_READ_WRITE, sizeof(cl_uint) * num_bins);
cl::Buffer buf_lut(context, CL_MEM_READ_WRITE, sizeof(cl_uchar) * num_bins);

```

Figure 6: Buffer creation

```

    ... is_color = (spectrum == 0),

    const int num_bins = 256;
    int size = width * height;
    vector<unsigned char> output(size * spectrum);

```

Figure 7: Configurable number of bins

```

// 2. Inclusive scan using Hillis-Steele
kernel void scan_kernel(__global uint* input, __global uint* output, int size) {
    int id = get_global_id(0);
    if (id >= size) return;
    output[id] = input[id];
    barrier(CLK_GLOBAL_MEM_FENCE);

    for (int offset = 1; offset < size; offset <= 1) {
        uint val = 0;
        if (id >= offset)
            val = output[id - offset];
        barrier(CLK_GLOBAL_MEM_FENCE);
        output[id] += val;
        barrier(CLK_GLOBAL_MEM_FENCE);
    }
}

```

Figure 8: Hillis-Steele algorithm used on the cumulative histogram

```

// OpenCL setup
vector<cl::Platform> platforms;
cl::Platform::get(&platforms);
cl::Platform platform = platforms.front();

vector<cl::Device> devices;
platform.getDevices(CL_DEVICE_TYPE_GPU, &devices);
cl::Device device = devices.front();

cl::Context context({ device });
cl::Program::Sources sources;

```

Figure 9: Platform and device selection

For visualisation purposes, I created a histogram and cumulative histogram graph from the data by converting it to a CSV file and then using Python and Matplotlib. In the histogram graph, it shows the frequency of pixel intensity values in the image. The tall, narrow spike at around 135 shows that the vast majority of pixels have mid-range brightness. The cumulative histogram graph shows the cumulative sum of pixel intensities across the image. It stays flat until around 120, which shows that there are very few dark pixels, and then there is a steep slope between ~130 and 170, which shows that there is a rapid accumulation of pixels, where most of the image information lies.

